

Computer Science COMP-3220 – Summer 2026

Assignment 1

Due: End of Sunday, June 14, 2026

Note: Submit your assignment as a **single pdf file** on Brightspace. The pdf file should be named Assignment1_StudentId.zip, which replaces *StudentId* with your university student number.

Total Mark: 40

Question 1 — Introduction to Design Patterns (13 marks)

Identifying Design Problems & Improving Flexibility

You are given the following Java code:

```
class PaymentProcessor {
    public void processPayment(String method) {
        if (method.equals("CREDIT")) {
            System.out.println("Processing credit card payment");
        } else if (method.equals("PAYPAL")) {
            System.out.println("Processing PayPal payment");
        } else if (method.equals("BITCOIN")) {
            System.out.println("Processing Bitcoin payment");
        }
    }
}
```

Tasks:

1. **Identify at least two design issues** in this code. (3 marks)
2. **Refactor the code to improve design quality** using *basic OO principles* (e.g., interfaces, polymorphism — no need to name a specific design pattern). (5 marks)
3. Briefly explain how your solution improves: (2 marks)
 - extensibility
 - maintainability
4. **Draw a UML Class Diagram** for your refactored solution. Include: (3 marks)
 - All classes/interfaces
 - Relationships (e.g., realization, dependency)

Question 2 — Observer Pattern (13 marks)

Implementing a Subscription System

You are given a partial system for a weather station:

```
interface Observer {
    void update(float temperature);
}
class WeatherStation {
```

```

private float temperature;

public void setTemperature(float temp) {
    this.temperature = temp;
    // TODO: notify observers
}
}

```

Tasks:

1. Complete the design using the **Observer Pattern**. (6 marks)
 - Add Subject interface
 - Maintain observers list
 - Implement register/remove/notify
2. Implement one concrete observer: PhoneDisplay. (2 marks)
3. Explain when Observer Pattern is preferred over polling. (2 marks)
4. **Draw a UML Class Diagram** for the Observer Pattern implementation. Include: (3 marks)
 - Subject, Observer
 - One concrete observer
 - Multiplicity between Subject and Observers

Question 3 — Decorator Pattern (14 marks)

Extending Behavior Dynamically

You are given a simple coffee system:

```

interface Coffee {
    double cost();
    String description();
}
class BasicCoffee implements Coffee {
    public double cost() { return 5.0; }
    public String description() { return "Basic Coffee"; }
}

```

Tasks:

1. Implement a Decorator base class. (3 marks)
2. Add two decorators:
 - Milk (+1.5)
 - Sugar (+0.5) (4 marks)
3. Show how to create:
 - Coffee with milk and sugar (1 mark)
4. Explain why inheritance is not suitable here. (2 marks)
5. **Draw a UML Class Diagram** for the Decorator Pattern. Include: (4 marks)
 - Component
 - Concrete component
 - Decorator base class
 - At least one concrete decorator